

Static Analysis

SonarQube

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	15	S2293	The diamond operation ("<>") should be used	Maintainability

```
private List<String> titles = new ArrayList<String>();
```

Correzione:

```
private List<String> titles = new ArrayList<>();
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	68	S1126	Return of boolean expression should not be wrapped into an "if-then-else" statement	Maintainability

```
if(accountActive)
    return true;
return false;
```

Correzione:

```
return this.isAccountActive();
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	74	S4973	Strings and Boxed types should be compared using "equals()"	Reliability

```
if (accountActive && this.userID == id)
```

Correzione:

```
if (accountActive && this.userID.equals(id))
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	82	S4973	Strings and Boxed types should be compared using "equals()"	Reliability

```
return u.userID == this.userID;
```

Correzione:

```
return u.userID.equals(this.userID);
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	86	S106	Standard outputs should not be used directly to log anything	Maintainability
User	86	S2629	"Preconditions" and logging arguments should not require evaluation	Maintainability

```
System.out.println("User Info: " + firstname + " " + lastname + "  
(Username: " + username + ")");
```

Correzione:

```
import java.util.logging.Logger;  
//...
```

```
private static final Logger logger =
    Logger.getLogger(User.class.getName());
//...
if(logger.isLoggable(Level.INFO)) {
    logger.info("User Info: " + firstname + " " + lastname + "
(Username: " + username + ")");
}
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	89	S112	Generic exceptions should never be thrown	Maintainability

```
public void linkCart(Cart cart) throws Exception{
    if(cart == null)
        throw new Exception();
    this.cart = cart;
}
```

Correzione:

```
public void linkCart(Cart cart) throws NullPointerException{
    if(cart == null)
        throw new NullPointerException();
    this.cart = cart;
}
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	100	S2116	"hashCode" and "toString" should not be called on array instances	Reliability

```
return roles.toString();
```

Correzione:

```
import java.util.Arrays;
// ...
return Arrays.toString(roles);
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	103	S100	Method names should comply with a naming convention	Maintainability

```
public void PrintEveryRole()
```

Correzione:

```
public void printEveryRole()
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	104	S2189	Loops should not be infinite	Reliability
User	104	S2251	A "for" loop update clause should move the counter in the right direction	Reliability

```
for (int i = roles.length; i>0; i++)
```

Correzione:

```
for (int i = roles.length-1; i>0; i--)
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
User	105	S106	Standard outputs should not be used directly to log anything	Maintainability

```
System.out.println(roles[i]);
```

Correzione:

```
logger.info(roles[i]);
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	17	S1124	Modifiers should be declared in the correct order	Maintainability
UsersManager	17	s115	Constant names should comply with a naming convention	Maintainability

```
public final static String basicUserID = "User00-";
```

Correzione:

```
public static final String BASIC_USER_ID = "User00-";
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	18	S1124	Modifiers should be declared in the correct order	Maintainability
UsersManager	18	s2386	Mutable fields should not be "public static"	Maintainability
UsersManager	18	s2293	The diamond operator ("<>") should be used	Maintainability

```
public final static List<User> users = new ArrayList<User>();
```

Correzione:

```
protected static final List<User> users = new ArrayList<>();
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	22	S6437	Credentials should not be hard-coded	Security

```
try (Connection conn =
    DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",
    BASIC_USER_ID+userID, "password");)
```

Correzione:

```
try (Connection conn =
    DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",
    BASIC_USER_ID+userID, System.getenv("DB_PASSWORD"));
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	24	S2095	Resources should be closed	Reliability

```
try (Connection conn =
    DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",
    BASIC_USER_ID+userID, "password");) {
    String query = "select firstname, lastname " + "from USERS where
    username="+ (BASIC_USER_ID+userID);
    PreparedStatement stmt = conn.prepareStatement(query);
    ResultSet rs = stmt.executeQuery();
    //...
}
```

Correzione:

```
String query = "select firstname, lastname " + "from USERS where
username="+ (BASIC_USER_ID+userID);
try (Connection conn =
    DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase",
    BASIC_USER_ID+userID, System.getenv("DB_PASSWORD"));
```

```
    PreparedStatement stmt = conn.prepareStatement(query);
    ResultSet rs = stmt.executeQuery();)
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	27	S2589	Boolean expressions should not be gratuitous	Maintainability

```
while (rs.next())
    if(rs != null)
        return true;
return false;
```

Correzione:

```
return (rs.next());
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	45	S5413	'List.remove()' should not be used in ascending 'for' loops	Maintainability

```
for (int i = 0; i < titles.size(); i++) {
    if (titles.get(i).isEmpty()) {
        titles.remove(i);
    }
}
```

Correzione:

```
titles.removeIf(String::isEmpty);
```

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	54	S2737	"catch" clauses should do more than rethrow	Maintainability

Classe	Linea	Codice errore	Messaggio errore	Tipo errore
UsersManager	54	S112	Generic exceptions should never be thrown	Maintainability

```
void addCartToUser(User user, Cart cart) throws Exception {
    try {
        user.linkCart(cart);
    } catch (Exception e) {
        throw e;
    }
}
```

Correzione:

```
void addCartToUser(User user, Cart cart) throws NullPointerException {
    user.linkCart(cart);
}
```

Hints

Classe	Linea	Messaggio errore
User	11	Field can be final

```
private String userID;
private String firstname;
private String lastname;
```

Correzione:

```
private final String userID;
private final String firstname;
private final String lastname;
```

Classe	Linea	Messaggio errore
User	86	Inefficient use of string concatenation in logger

```
logger.info("User Info: " + firstname + " " + lastname + " (Username: "
+ username + ")");
```

Correzione:

```
logger.info(String.format("User Info: %s %s (Username: %s)", firstname,
lastname, username));
```

Classe	Linea	Messaggio errore
UsersManagement	36	Method is never used

```
void addUser(User user) throws SQLException
//...
void removeEmptyTitlesFromUser(User user)
//...
void addCartToUser(User user, Cart cart) throws Exception
```

Correzione:

```
public void addUser(User user) throws SQLException
//...
public void removeEmptyTitlesFromUser(User user)
//...
public void addCartToUser(User user, Cart cart) throws
NullPointerException
```

DesigniteJava

1. *Identificare i metodi con Complessità ciclomatica più alta nell'intero progetto "assignment"*

Da `MethodMetrics.csv` :

I metodi sono `Cart.updateProductQuality` e

`UsersManager.removeEmptyTitlesFromUser` , entrambi con complessità ciclomatica uguale a 3.

2. *Elencare e spiegare i code smell presenti nelle classi User e UserManager*

Da `ImplementationSmells.csv` :

Classe	Metodo	Smell		Descrizione
User	User	Long Parameter List	Il metodo ha 7 parametri.	Se il costruttore di una classe ha tanti parametri, questa potrebbe star violando il principio di responsabilità singola (SRP). La classe <code>User</code> tratta diversi aspetti relativi agli utenti, che potrebbero essere invece gestiti in varie classi, ciascuna trattante funzioni più specifiche. Inoltre, l'elevata quantità di argomenti rende il codice meno leggibile, si potrebbero introdurre valori di default per i campi della classe e metodi set per alleggerire il metodo.
UsersManager	findUserFromDB	Long Statement	La lunghezza dello statement <code>try(Connection conn=[...])</code> è 229.	Statement troppo lunghi complicano leggibilità e manutenzione del codice. In questo caso si potrebbero dividere in più fasi le operazioni svolte sul DB, o evitare di usare un try-with-resources

3. La classe `UserManager` viola qualche principio di buon design? Se sì spiegare perché

Da `DesignSmells.csv` :

Smell	Principio violato	Spiegazione
Unutilized Abstraction	Semplicità	La classe è inutilizzata. Normalmente un problema, in questo caso si può ignorare perché le classi non sono parte di un sistema funzionante, ma solo prese fuori contesto a scopo didattico
Deficient Encapsulation	Astrazione	I campi <code>BASIC_USER_ID</code> e <code>users</code> hanno visibilità pubblica. Sarebbe meglio se i campi fossero privati e accessibili con appositi metodi

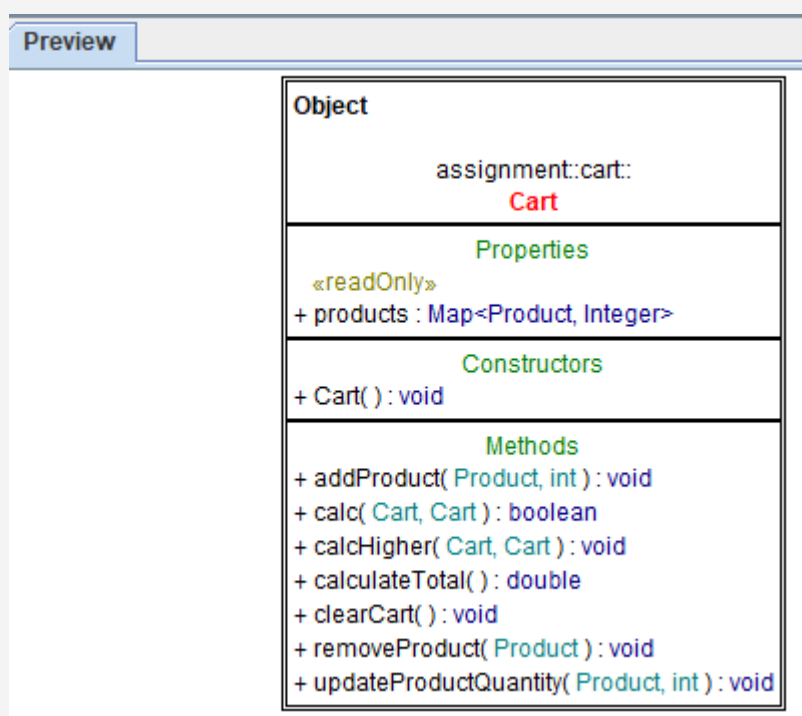
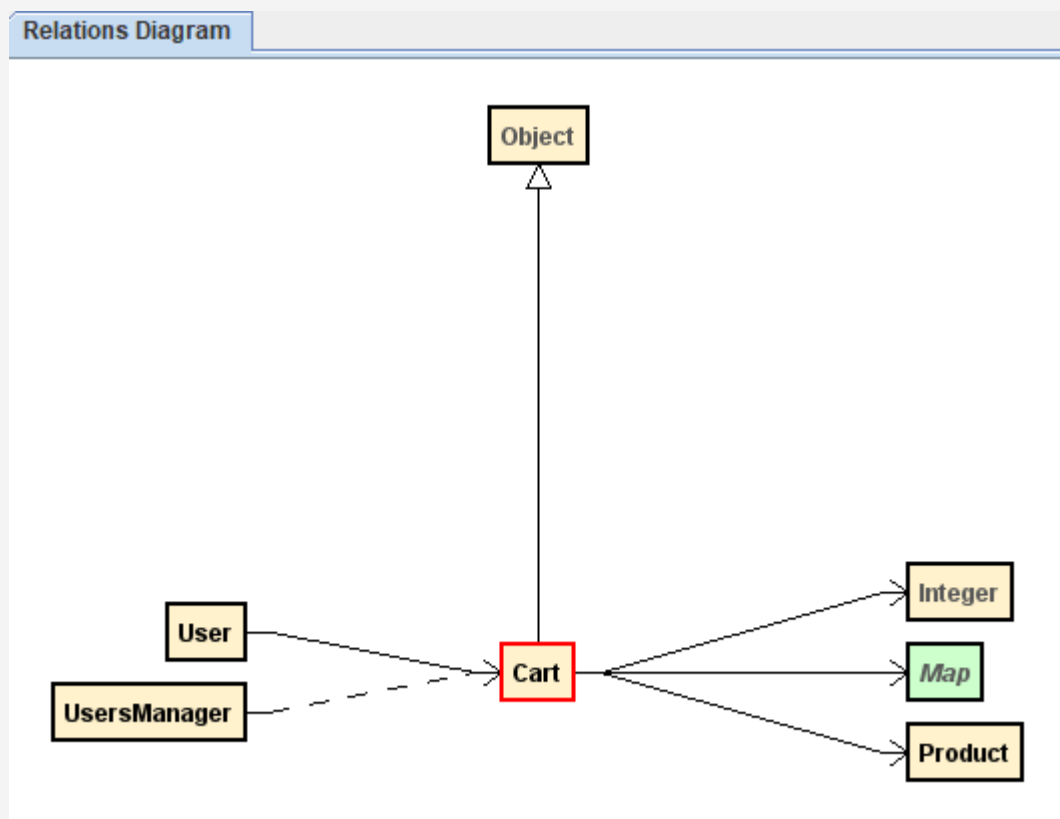
4. Esistono problemi a livello di Architettura software di questo progetto? Se si spiegare quali

Da `ArchitectureSmells.csv` :

Il tool non ha rilevato alcun problema.

Class Visualizer

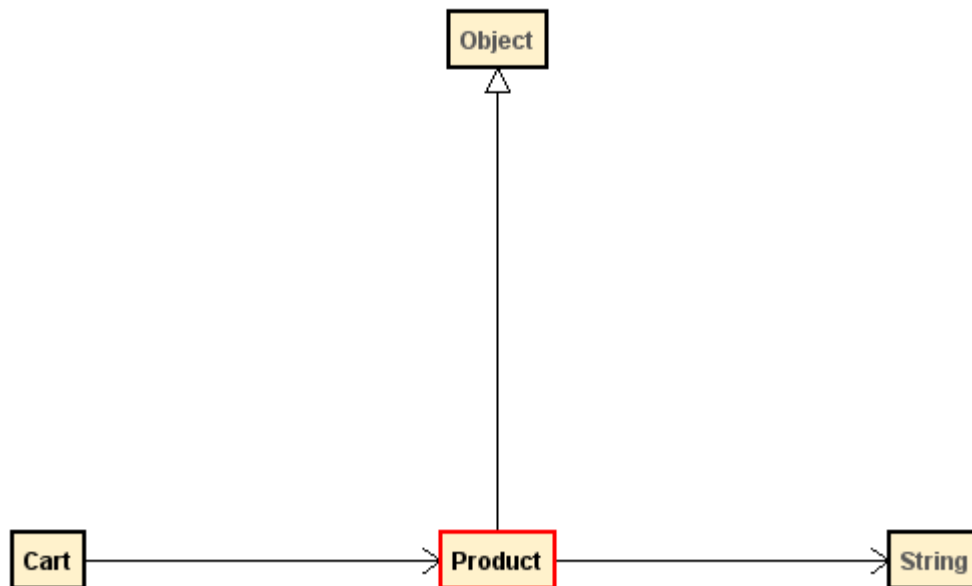
Cart



Uses		Used By	
3	Integer, Map, Product	2	User, UsersManager

Product

Relations Diagram

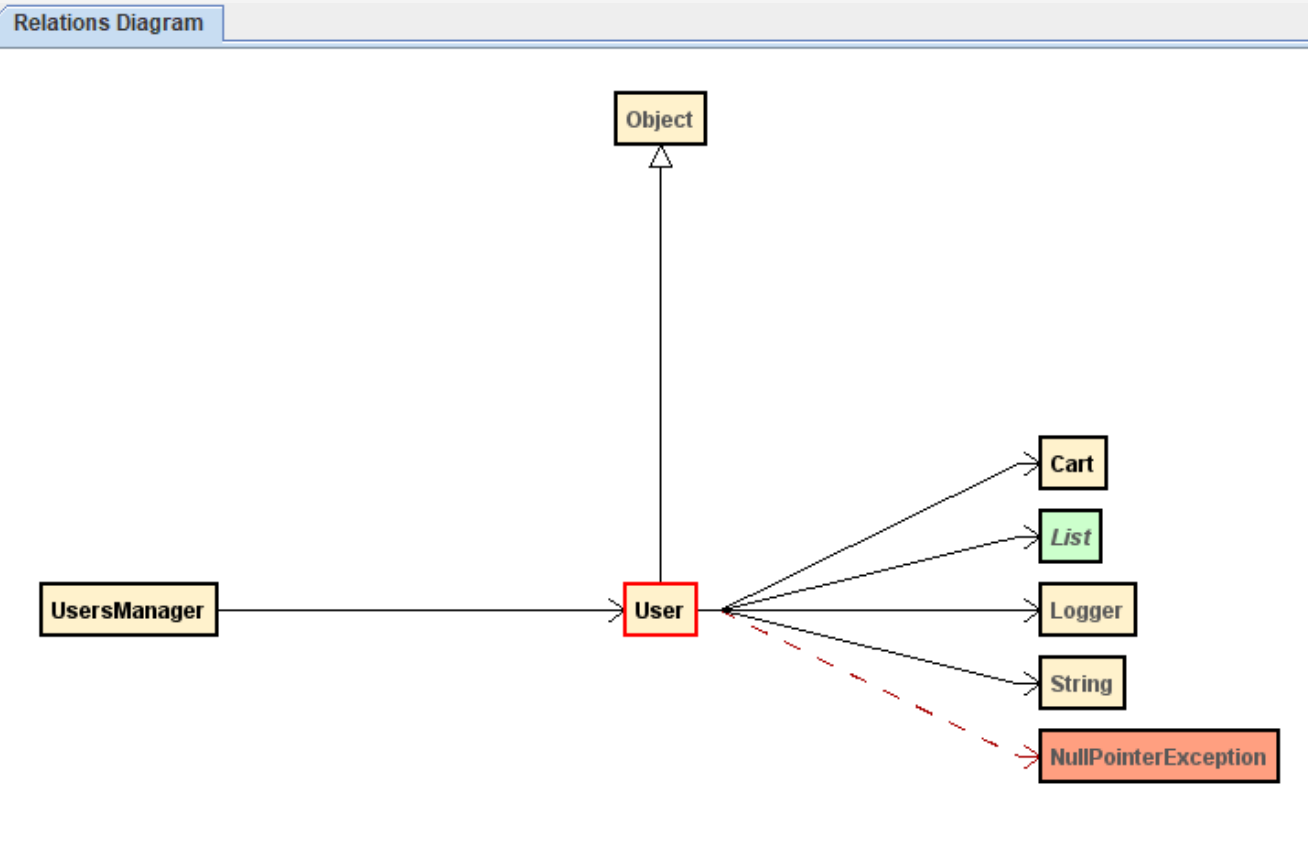


Preview

Object	
	assignment::cart:: Product
Fields	
- isAvailable : boolean	
Properties	
+ brand : String	
+ category : int	
+ description : String	
+ id : String	
+ name : String	
+ stockQuantity : int	
+ unitPrice : double	
Constructors	
+ Product(String, String, String, double, int) : void	
Methods	
+ IsAvailable() : boolean	
+ setFullDetailedProduct(String, String, String, double, int, String, int, boolean) : void	
+ setIsAvailable(boolean) : void	

Uses		Used By	
1	String	1	Cart

User

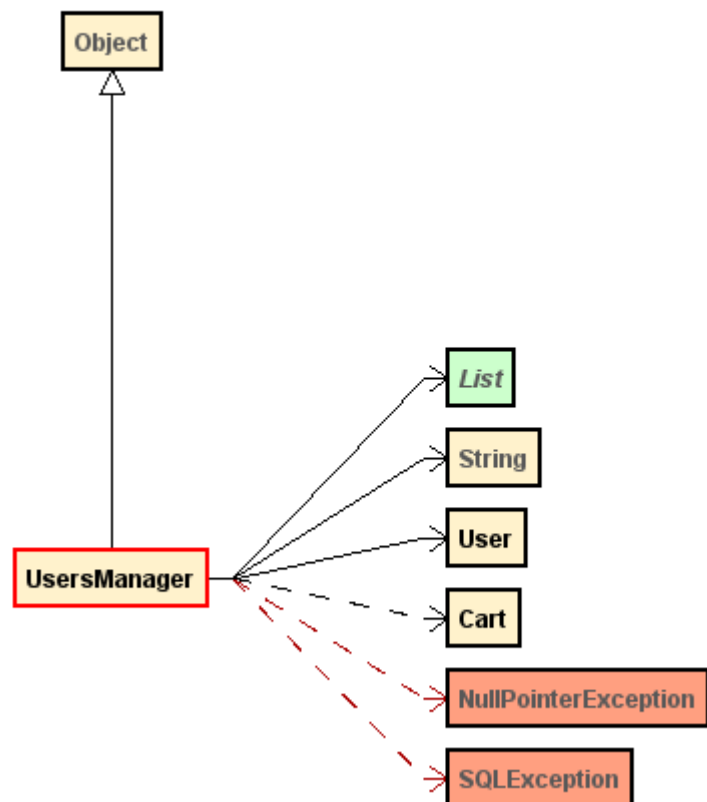


Object	
assignment::user:: User	
Constants	
- logger : Logger	
Fields	
- roles : String[]	
Properties	
+ accountActive : boolean «readOnly»	
+ active : boolean «readOnly»	
+ cart : Cart «final» «readOnly»	
+ firstname : String «final» «readOnly»	
+ lastname : String «readOnly»	
+ titles : List<String> «final» «readOnly»	
+ userID : String	
+ username : String	
Constructors	
+ User(String, String, String, String, boolean, List<String>, String[]) : void	
Methods	
+ deactivateAccount(String) : boolean	
+ isEqual(User) : boolean	
+ linkCart(Cart) : void	
+ printAllRoles() : String	
+ printEveryRole() : void	
+ printUserInfo() : void	
+ updateUsername(String) : void	

Uses		Used By	
4	Cart, List, Logger, String	1	UsersManager

UsersManager

Relations Diagram



Preview

Object

assignment::user::
UsersManager

Constants

+ BASIC_USER_ID : String
users : List<User>

Constructors

+ UserManager() : void

Methods

+ addCartToUser(User, Cart) : void
+ addUser(User) : void
+ findUserFromDB(String) : boolean
+ removeEmptyTitlesFromUser(User) : void

Uses		Used By	
4	List, String, User, Cart	0	